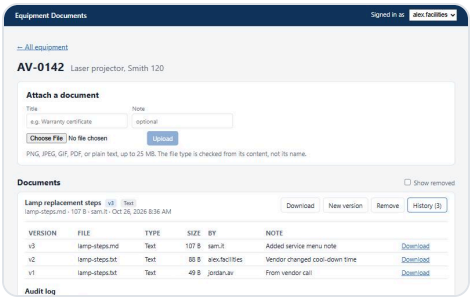
Chase Moses

Software Engineer · C#, .NET, Blazor, SQL Server · BYU-Idaho alumnus, Rexburg

chsmoses@outlook.com LinkedIn (https://linkedin.com/in/chase-moses) GitHub (https://github.com/chasemoses)

Each project below comes from a system I've built or worked on. The work projects are rebuilt from scratch so the code can be public, and the client and personal projects are cleaned up for the same reason. Every repository has a README that explains the decisions, the trade-offs, and what testing caught.



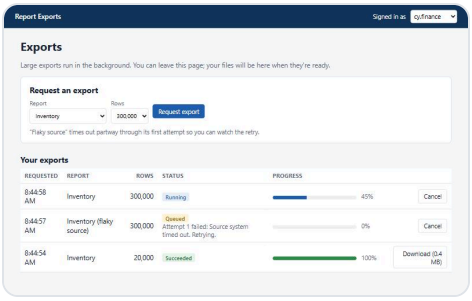
Versioned Document Attachments

Rebuilt from a document management system I designed and built at FileOnQ

Attach warranties, manuals, and photos to campus equipment, with full version history. Uploads are checked by their actual content, so a renamed `.exe` is rejected. They're streamed to disk while being hashed, and identical files are stored once. Two people uploading at the same moment can't overwrite each other, and removing a document is a soft delete with a required reason and an audit trail.

C# · Blazor Server · Entity Framework Core · SQL Server · ASP.NET Core · Secure file handling · Optimistic concurrency · xUnit

Repository and design notes (https://github.com/chasemoses/VersionedDocuments)



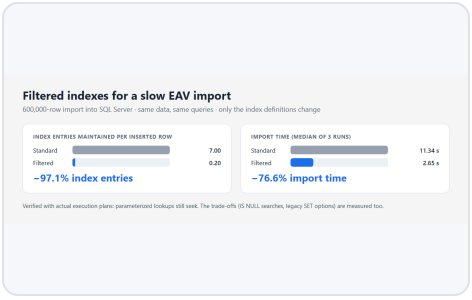
Background Report Exports

Rebuilt from background download processing I built at FileOnQ

Large exports move out of the web request. The page responds immediately while a hosted worker builds the file and reports live progress. The queue lives in SQL Server, so a restart in the middle of an export resumes it instead of losing it. Temporary failures retry with backoff, users can cancel, and only the person who asked for a file can download it.

ASP.NET Core hosted services · Blazor Server · Entity Framework Core · SQL Server · Concurrency and cancellation · Authentication and authorization · xUnit

Repository and design notes (https://github.com/chasemoses/BackgroundExports)



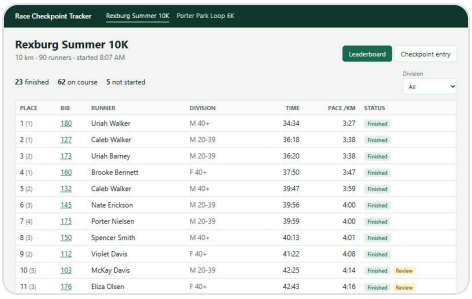
Filtered Indexes for a Slow Import

Reproduces a performance fix I made at FileOnQ

A bulk import was slow because SQL Server maintained seven index entries for every row, mostly for empty values. Filtered indexes cut that to 0.20 entries per row and the import time by 77%. The benchmark checks the actual execution plans to confirm searches still use the indexes, and it shows the cases that get worse so the trade-off is a decision, not a surprise.

SQL Server indexing · Execution plans · Performance investigation · C# / ADO.NET · SqlBulkCopy · Technical writing

Repository and results (https://github.com/chasemoses/EavFilteredIndexes)



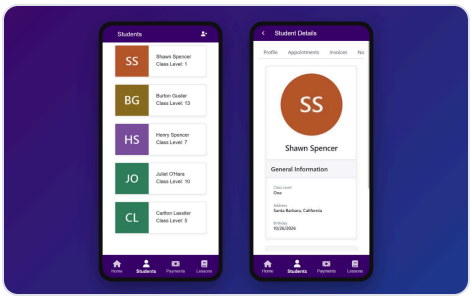
Race Checkpoint Tracker

Rebuilt from a Python app I wrote for the City of Rexburg

Volunteers type bib numbers at checkpoints, and the app turns those entries into live results for point-to-point and multi-lap courses. It works out which lap each crossing belongs to, ignores double taps, and flags finishers who missed a checkpoint for review instead of disqualifying them. Testing in the browser caught a time-zone bug that every unit test missed, and it now has a regression test.

Angular · TypeScript · RxJS · ASP.NET Core REST API · Entity Framework Core · SQL Server · xUnit integration tests

Repository and design notes (https://github.com/chasemoses/RaceCheckpointTracker)



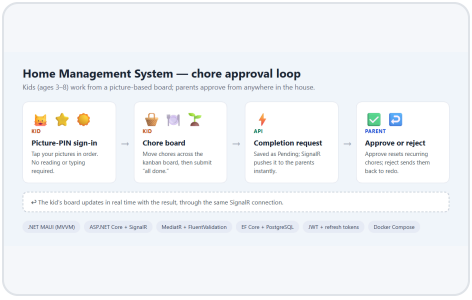
Wisdom in Learning

Client project as sole developer · built 2023, rewritten 2024

Student management for a company teaching students in China: students and families, class levels, payments, and invoice history on Windows, Android, and iOS. It follows Clean Architecture behind a REST API. The 2024 rewrite moved the interface into shared Blazor components hosted in .NET MAUI, so one codebase gives a desktop sidebar and a mobile tab bar. The README is candid about what I'd do differently today.

.NET MAUI Blazor Hybrid · Razor component libraries · ASP.NET Core Web API · Entity Framework Core · SQL Server · Clean Architecture · MediatR

Repository and architecture notes (https://github.com/chasemoses/WisdomInLearning)



Home Management System

Personal project · in active development

A self-hosted chore app for my family. Kids ages 3–8 sign in by tapping pictures and work from a picture-based board, and parents approve finished chores from their phones in real time. It was planned and built in verified phases with AI assistance. The repository keeps the requirements, research, and plans, including a feasibility spike that deliberately said no to fingerprint sign-in for kids.

.NET MAUI (MVVM) · ASP.NET Core · SignalR · Entity Framework Core · PostgreSQL · Docker Compose · JWT and refresh tokens · MediatR and FluentValidation

Repository and planning history (https://github.com/chasemoses/HomeManagementSystem)

Background

Now	Software Developer at FileOnQ (since 2024): full-stack C#/.NET, Blazor, Entity Framework Core, SQL Server, in two-week sprints
Before	IT Support Technician, City of Rexburg; Tier 2 technical support, M9 Creations
Education	B.S. Computer Software Engineering, Brigham Young University–Idaho, 2023
AI tools	GitHub Copilot daily at work; Claude Code for these projects. Each README notes how AI was used and what I checked myself.